

GPUMODE Lecture Study Notes: Lecture 82

Helion: A High-Level DSL for ML Kernels

Core Problem the Lecture Addresses

This lecture introduces **Helion**, a high-level Python-embedded domain-specific language for authoring machine-learning kernels. The core problem is that modern GPU programmers are often forced into an uncomfortable choice:

- **PyTorch and `torch.compile`:** easy to write, easy to integrate into models, and capable of automatic graph-level optimization, but the user has limited control over the exact generated kernel.
- **Triton, CUDA, CuTe, CUTLASS, and low-level DSLs:** powerful and precise, but they require manual work for indexing, masks, launch grids, tiling, program IDs, memory layout, cache hints, and backend configuration.

Helion tries to occupy the middle of this design space. It is lower level than PyTorch because the programmer explicitly controls what belongs inside one kernel. It is higher level than Triton because the programmer writes PyTorch-like tensor operations over tiles, while Helion and its auto-tuner search over many low-level implementation details.

The lecture’s main technical claim is:

PyTorch-like tiled semantics+integrated auto-tuning $\approx$ low-effort kernels with competitive performance.

The motivating use case comes from users of `torch.compile`. Many users liked automatic compilation, but wanted more direct control over generated kernels. Helion is a response to that demand: it gives explicit kernel authoring while preserving much of the programming style of PyTorch.

The key mental model is: Helion is “PyTorch with tiles” plus a serious auto-tuner. The programmer expresses what computation should happen over tiled regions, and the system searches over how that tiled computation should be implemented.

Required Background

GPU Kernel Execution Model

A GPU kernel is launched over a grid of programs or thread blocks. In CUDA language, the hierarchy is:

$$\text{grid} \rightarrow \text{thread blocks} \rightarrow \text{warps} \rightarrow \text{threads}.$$

In Triton, the analogous unit is often called a **program**. Each program usually maps to a block of work. A program is identified by one or more program IDs, such as:

$$p_m = \text{tl.program_id}(0), \quad p_n = \text{tl.program_id}(1).$$

These IDs are then used to compute offsets into tensors. For a two-dimensional tiled matrix multiplication, a Triton kernel might compute tile origins as:

$$m_0 = p_m B_M, \quad n_0 = p_n B_N,$$

where B_M and B_N are block sizes for the row and column dimensions.

Why Manual Kernel Authoring Is Tedious

In Triton, a kernel author often manually handles:

- pointer arithmetic;
- block pointer or tensor descriptor construction;
- launch grid calculation;
- masks for ragged boundary tiles;
- block sizes and reduction block sizes;
- ordering and swizzling of program IDs;
- persistent launch strategies;
- backend flags such as number of warps, number of stages, and warp specialization;
- cache and eviction policies;
- separate launcher code and device kernel code.

For example, even a simple vector add requires computing offsets, creating masks, loading from memory, adding, and storing:

```

@triton.jit
def add_kernel(x, y, out, n_elements, BLOCK_SIZE: tl.constexpr):
    pid = tl.program_id(0)
    offsets = pid * BLOCK_SIZE + tl.arange(0, BLOCK_SIZE)
    mask = offsets < n_elements
    x_val = tl.load(x + offsets, mask=mask)
    y_val = tl.load(y + offsets, mask=mask)
    tl.store(out + offsets, x_val + y_val, mask=mask)

```

This is not conceptually hard, but it is repetitive. Helion’s goal is to let the user write the computation in a form closer to:

```

for tile in hl.tile(numel):
    out[tile] = x[tile] + y[tile]

```

The tile describes a block of the iteration space. Helion then lowers the tiled computation to a Triton kernel and uses auto-tuning to search over the low-level details.

What a DSL Must Balance

A GPU DSL must balance four goals:

1. **Productivity:** the code should be easy to write and understand.
2. **Performance:** generated kernels should approach hand-written Triton or CUDA performance.
3. **Portability:** kernels should adapt to new GPU architectures without full rewrites.
4. **Control:** expert users should still be able to influence schedules, memory behavior, and low-level instructions.

Helion’s design chooses high-level semantics and integrated auto-tuning as the main path to performance portability.

Main Concepts

Helion as a Layer Between PyTorch and Triton

Helion sits between PyTorch and Triton:

PyTorch → Helion → Triton → GPU machine code.

Compared with PyTorch, Helion gives more direct control because a Helion kernel corresponds to one fused generated kernel. Compared with Triton, Helion hides much of the indexing and launch boilerplate.

The lecture emphasizes the invariant:

One Helion kernel lowers to one Triton kernel.

This is central to understanding Helion. Unlike `torch.compile`, Helion does not decide graph fusion boundaries for the user. The user writes a Helion kernel, and the contents of the device region become one fused kernel.

Host Region and Device Region

A Helion kernel has two conceptual regions:

- **Host region:** ordinary Python and PyTorch code that runs eagerly on the host.
- **Device region:** code inside the `hl.tile` loop that is compiled into a Triton kernel.

A typical Helion kernel therefore has the structure:

```
@helion.kernel
def kernel(x, y):
    # Host region: ordinary Python or PyTorch.
    out = torch.empty_like(torch.broadcast_tensors(x, y)[0])

    # Device region: compiled into one Triton kernel.
    for tile in hl.tile(out.numel()):
        out[tile] = x[tile] + y[tile]

    # Host region again.
    return out
```

The host region may allocate tensors, compute shapes, perform assertions, and run eager Python logic. The device region describes tiled computation.

`hl.tile`: The Central Abstraction

The main language construct is:

```
hl.tile(size).
```

It subdivides an iteration space into tiles. The exact subdivision is not fixed in the source code. Instead, it creates a search space for the auto-tuner.

For a one-dimensional problem of size N , Helion conceptually creates tiles:

$$[0, B), [B, 2B), [2B, 3B), \dots,$$

where B is a block size chosen by the auto-tuner. For a two-dimensional iteration space, Helion may tile over rows and columns:

$$(i, j) \in [0, M) \times [0, N),$$

using tile sizes B_M and B_N .

The programmer writes:

```
for tile_m, tile_n in hl.tile((M, N)):
    out[tile_m, tile_n] = ...
```

The auto-tuner may then search over:

B_M , B_N , iteration order, flattening, swizzling, program-ID layout.

PyTorch Operations Inside Tiles

One major design choice is that Helion allows many PyTorch operations inside the device region. Instead of requiring a separate Helion operator library, Helion reuses the PyTorch compiler stack, especially TorchInductor, to lower PyTorch operations to Triton.

For example, a Helion kernel can use operations such as:

- `torch.add`;
- `torch.where`;
- `torch.mean`;
- `torch.rsqrt`;
- `torch.softmax`;
- matrix multiplication operations such as `addmm` or equivalent tiled matmul logic.

This is one of the main reasons Helion feels closer to PyTorch than to Triton.

Auto-Tuning as Part of the Language Identity

The speakers emphasize that Helion is not merely Triton with syntax sugar. Its core identity includes the integrated auto-tuner.

A single Helion kernel defines a search space containing many possible generated Triton kernels. The tuner may search thousands of candidates. The lecture mentions a typical auto-tuning run of about 1200 candidate kernels taking around 20 minutes for a full search.

The central trade-off is:

human effort $\longleftrightarrow$ machine search effort.

Instead of manually writing and benchmarking many Triton variants, the programmer writes one Helion kernel and lets the auto-tuner search over implementation choices.

Helion Configuration Space

Indexing Modes

Triton supports multiple indexing styles:

1. **Pointer arithmetic:** explicit offsets added to base pointers.
2. **Block pointers:** structured block-level memory access.
3. **Tensor descriptors:** newer structured access mechanisms for recent NVIDIA hardware.

In Triton, switching between these modes often requires rewriting the kernel. In Helion, the user writes PyTorch-style indexing:

$$x[\text{tile}] \quad \text{or} \quad x[\text{tile}_m, \text{tile}_n].$$

The auto-tuner can choose between pointer indexing, block pointers, and tensor descriptors. The lecture notes that different indexing modes may win for different shapes. For example, tensor descriptors may be faster on large inputs, while pointer indexing may be faster on smaller inputs.

Block Sizes

In Triton, block sizes are often explicit compile-time constants:

```
BLOCK_M: tl.constexpr
BLOCK_N: tl.constexpr
BLOCK_K: tl.constexpr
```

The kernel author manually threads these constants through indexing logic and launch-grid computation. In Helion, an `hl.tile` loop implicitly creates block-size choices.

For a two-dimensional tile loop:

```
hl.tile((M,N))
```

the tuner may choose:

$$B_M \in \mathcal{B}_M, \quad B_N \in \mathcal{B}_N.$$

The number of programs in each dimension is approximately:

$$G_M = \left\lceil \frac{M}{B_M} \right\rceil, \quad G_N = \left\lceil \frac{N}{B_N} \right\rceil.$$

The total number of programs is:

$$G = G_M G_N.$$

Loop Flattening

Helion can flatten a multi-dimensional iteration space into a one-dimensional program space. For example, a two-dimensional logical grid:

$$(p_m, p_n) \in [0, G_M) \times [0, G_N)$$

can be mapped to a single flattened program ID:

$$p = p_m G_N + p_n.$$

The inverse mapping is:

$$p_m = \left\lfloor \frac{p}{G_N} \right\rfloor, \quad p_n = p \bmod G_N.$$

Flattening changes the ordering in which tiles are assigned to programs. This can affect locality, scheduling, and cache behavior. In a low-level language, exploring this variant manually is tedious. In Helion, it is another auto-tunable option.

Loop Reordering and Swizzling

Tile order matters because it changes how neighboring programs reuse cache lines and L2 data. For example, if several adjacent row tiles use the same column tile of a matrix, a grouping strategy may improve L2 reuse.

Suppose a tiled matrix multiplication computes:

$$C_{m,n} = \sum_{k=0}^{K-1} A_{m,k} B_{k,n}.$$

A naive ordering over (m, n) may be:

$$(0, 0), (0, 1), (0, 2), \dots, (1, 0), (1, 1), \dots$$

A grouped ordering might process several m blocks for the same n block, or several n blocks for the same m block, depending on which operand should remain hot in cache.

Helion can auto-tune these grouping and reordering strategies.

Reduction Rolling

A reduction such as:

$$y_i = \sum_{j=0}^{N-1} x_{i,j}$$

can be written naively by loading the entire row into one tile and reducing it. However, if N is large, this creates too many live values and may exceed available registers or per-program capacity.

A rolled reduction processes the row in chunks:

$$a_i^{(0)} = 0,$$

$$a_i^{(t+1)} = a_i^{(t)} + \sum_{j=tB}^{(t+1)B-1} x_{i,j},$$

and after all chunks:

$$y_i = a_i^{(T)}.$$

Here B is a reduction block size and $T = \lceil N/B \rceil$. Helion can automatically convert certain direct reductions into rolled reductions, introducing another tunable block size.

Program ID Types and Persistent Grids

Helion can tune how logical tiles map to program IDs. Options include:

- one-dimensional launch grids;
- two-dimensional launch grids;
- persistent launch grids.

In a persistent launch grid, the kernel may launch roughly one program per SM, and each program loops over many virtual work items. If the GPU has S SMs, a persistent strategy might launch S programs and map virtual tile IDs by:

$$v = p + tS,$$

where p is the physical program ID and t is the loop iteration inside that persistent program. This can improve scheduling and pipelining for certain workloads.

Backend Flags

Helion also tunes or passes through Triton backend settings such as:

- number of warps;
- number of pipeline stages;
- warp specialization;
- other backend-specific code generation flags.

These parameters affect occupancy, register pressure, memory pipelining, instruction scheduling, and latency hiding.

Automatic Masking

Boundary tiles often extend beyond tensor shapes. In Triton, the programmer usually writes explicit masks:

```
mask = offsets < n_elements
x_val = tl.load(x + offsets, mask=mask, other=0.0)
tl.store(out + offsets, y_val, mask=mask)
```

Helion automates much of this masking. The source code can index tiles directly, and Helion generates masks where needed. When shapes or layouts allow masks to be proven unnecessary, Helion can optimize them away.

Auto-Tuning Algorithm

Search Space

A Helion kernel defines a family of possible generated Triton kernels:

$$\mathcal{K} = \{K(c) \mid c \in \mathcal{C}\},$$

where c is a configuration and $\mathcal{C}$ is the configuration space. A configuration may include:

$c = (B_M, B_N, B_K, \text{indexing mode, flattening, program ID strategy, num warps, num stages, eviction policy}, \dots)$

The tuner wants:

$$c^* = \arg \min_{c \in \mathcal{C}} T(K(c), \text{shape, hardware}),$$

where T is measured runtime.

Current Algorithm

The lecture describes a simple but empirically effective algorithm:

1. Randomly sample an initial population of about 100 configurations.
2. Benchmark these candidates.
3. Select the top 5 fastest configurations.
4. Hill-climb from each selected configuration toward a local optimum.
5. Choose the fastest local optimum.

In pseudocode:

```
def autotune(config_space, benchmark):
    population = random_sample(config_space, n=100)
    scored = [(benchmark(c), c) for c in population]
    top = select_fastest(scored, k=5)

    local_minima = []
    for _, c in top:
        c_local = hill_climb(c, config_space, benchmark)
        local_minima.append((benchmark(c_local), c_local))

    return select_fastest(local_minima, k=1)[0]
```

The algorithm assumes that many kernel search spaces are reasonably regular. That is, nearby configurations tend to have related performance. This makes hill climbing useful.

Auto-Tuning Cost

The lecture mentions a full auto-tuning search taking around 20 minutes for about 1200 candidate kernels. This is expensive, but the intended workflow is:

1. Write a Helion kernel.
2. Run the auto-tuner for representative shapes and hardware.

3. Copy the best configuration into the source code.
4. Reuse the chosen configuration in later runs without re-tuning.

The amortized cost becomes reasonable if the tuned kernel is used many times.

Single Configuration Deployment

The simplest deployment mode is to tune once and paste the best configuration into the kernel. Future invocations use that configuration directly:

```
best_config = helion.Config(
    block_sizes=[...],
    indexing="tensor_descriptor",
    num_warps=4,
    num_stages=3
)

@helion.kernel(config=best_config)
def my_kernel(x):
    ...
```

This works well when the workload has a stable shape distribution.

Multiple Configurations for Shape Families

Different shapes may require different configurations. For example:

$$c_{\text{small}} = \arg \min_c T(c, N_{\text{small}}), \quad c_{\text{large}} = \arg \min_c T(c, N_{\text{large}}).$$

Helion can accept multiple configurations and test them for previously unseen shapes. It then caches the fastest choice for that shape.

```
configs = [small_shape_config, large_shape_config]
```

```
@helion.kernel(configs=configs)
def kernel(x):
    ...
```

Manual Routing

For more control, the programmer can manually route shapes to configurations:

```
def run(x):
    if x.numel() < 4096:
        return small_runner(x)
    else:
        return large_runner(x)
```

This is useful when the shape distribution is known and the cost of runtime testing should be avoided.

Hardware-Level Explanation

What Helion Hides and What It Exposes

Helion hides many details:

- exact program ID arithmetic;
- boundary masks;
- block pointer construction;
- launch-grid boilerplate;
- many backend configuration choices.

However, it still exposes the important high-level scheduling decision:

Which dimensions are parallel? Which dimensions are sequential?

This was emphasized strongly in the attention-kernel walkthrough. The outermost `hl.tile` loop maps to the parallel launch grid. Inner `hl.tile` loops or ordinary loops are sequential within a program.

Outer Tile Loop as Parallel Grid

For a kernel:

```
for tile_m, tile_n in hl.tile((M, N)):
    ...
```

the outer `hl.tile` loop represents parallel work. Conceptually, the generated Triton kernel has a launch grid like:

$$G_M = \left\lceil \frac{M}{B_M} \right\rceil, \quad G_N = \left\lceil \frac{N}{B_N} \right\rceil.$$

Each program handles one tile:

$$[p_m B_M, (p_m + 1) B_M) \times [p_n B_N, (p_n + 1) B_N).$$

Inner Loops as Sequential Work

An inner tile loop is sequential inside the generated program. This matters for reductions. For example, in attention, the output for a query block depends on a reduction over key positions. That key dimension cannot be naively parallelized without additional synchronization or atomics.

If the attention score is:

$$S = QK^T,$$

and softmax is applied over the key dimension:

$$P_{i,j} = \frac{\exp(S_{i,j})}{\sum_{\ell} \exp(S_{i,\ell})},$$

then for each query row i , the denominator depends on all key positions ℓ . Therefore, a single program often iterates sequentially over key blocks while maintaining running softmax statistics.

Occupancy and Register Pressure

A program that loads too much data into registers may reduce occupancy or spill to local memory. Occupancy is roughly constrained by resources per SM:

$$\text{active programs per SM} \leq \min \left(\left\lfloor \frac{R_{\text{SM}}}{R_{\text{program}}} \right\rfloor, \left\lfloor \frac{S_{\text{SM}}}{S_{\text{program}}} \right\rfloor, P_{\text{max}} \right),$$

where:

- R_{SM} is the register budget per SM;
- R_{program} is registers used per program;
- S_{SM} is shared memory per SM;
- S_{program} is shared memory used per program;
- P_{max} is the architectural limit on active programs.

Naive reductions that load an entire row can make R_{program} too large. Reduction rolling reduces per-program register usage by processing chunks.

Memory Hierarchy

The performance of Helion-generated kernels still depends on the usual GPU memory hierarchy:

$$\text{HBM} \rightarrow \text{L2 cache} \rightarrow \text{L1/shared memory} \rightarrow \text{registers}.$$

Important optimization questions include:

- Are global memory accesses coalesced?
- Is data reused from cache or reloaded from HBM?
- Are intermediate values kept in registers?
- Does the block size create too much register pressure?
- Does the schedule expose enough parallelism to hide latency?

Helion does not remove these hardware realities. It tries to search over implementation choices so that the generated Triton kernel lands in a good region of the performance space.

Cache Eviction Policies

The RMSNorm example discusses explicit eviction policies. Suppose a tensor value x is used twice:

$$r = \sqrt{\frac{1}{N} \sum_j x_j^2 + \epsilon}, \quad y_j = \frac{x_j}{r} w_j.$$

The input x_j may be useful after computing the denominator, while weight w_j may be used only once. Therefore:

- load x with an `evict_last` policy when it will be reused;
- load w with an `evict_first` policy when it has no future reuse.

This is a hardware-level hint to improve cache behavior.

Mathematical Reasoning

Arithmetic Intensity

A kernel’s arithmetic intensity is:

$$I = \frac{\text{FLOPs}}{\text{bytes moved}}.$$

The roofline model predicts achievable performance as:

$$P_{\text{achieved}} \leq \min(P_{\text{peak}}, B_{\text{mem}} I),$$

where:

- P_{peak} is peak compute throughput;

- B_{mem} is memory bandwidth;
- I is arithmetic intensity.

If $B_{\text{mem}}I < P_{\text{peak}}$, the kernel is memory-bound. If $P_{\text{peak}} < B_{\text{mem}}I$, it is compute-bound.

Matrix Multiplication

For matrix multiplication:

$$C = AB,$$

with:

$$A \in \mathbb{R}^{M \times K}, \quad B \in \mathbb{R}^{K \times N}, \quad C \in \mathbb{R}^{M \times N},$$

the elementwise formula is:

$$C_{m,n} = \sum_{k=0}^{K-1} A_{m,k} B_{k,n}.$$

The total FLOP count is approximately:

$$\text{FLOPs} = 2MNK.$$

A tiled implementation computes an output block:

$$C_{M_b, N_b} = \sum_{K_b} A_{M_b, K_b} B_{K_b, N_b}.$$

If tile sizes are B_M , B_N , and B_K , each inner step reads:

$$B_M B_K + B_K B_N$$

input elements and performs:

$$2B_M B_N B_K$$

FLOPs. The tile-level arithmetic intensity is approximately:

$$I_{\text{tile}} = \frac{2B_M B_N B_K}{s(B_M B_K + B_K B_N + B_M B_N)},$$

where s is bytes per element and the $B_M B_N$ term accounts for output traffic. Larger tiles usually improve reuse but may increase register pressure and reduce occupancy.

RMSNorm

RMSNorm normalizes a vector $x \in \mathbb{R}^N$ using:

$$\text{RMS}(x) = \sqrt{\frac{1}{N} \sum_{j=0}^{N-1} x_j^2 + \epsilon}.$$

The output is:

$$y_j = \frac{x_j}{\text{RMS}(x)} w_j.$$

Equivalently:

$$y_j = x_j \cdot \text{rsqrt} \left(\frac{1}{N} \sum_{\ell=0}^{N-1} x_\ell^2 + \epsilon \right) \cdot w_j.$$

The reduction term is:

$$s = \sum_{\ell=0}^{N-1} x_\ell^2.$$

A rolled reduction computes:

$$s^{(0)} = 0,$$

$$s^{(t+1)} = s^{(t)} + \sum_{\ell=tB}^{\min((t+1)B, N)-1} x_\ell^2.$$

This makes the per-step working set size proportional to B instead of N .

Attention

Scaled dot-product attention is:

$$O = \text{softmax} \left(\frac{QK^T}{\sqrt{d}} \right) V,$$

where:

$$Q \in \mathbb{R}^{M \times d}, \quad K \in \mathbb{R}^{N \times d}, \quad V \in \mathbb{R}^{N \times d_v}.$$

The score matrix is:

$$S_{i,j} = \frac{1}{\sqrt{d}} \sum_{r=0}^{d-1} Q_{i,r} K_{j,r}.$$

The softmax probabilities are:

$$P_{i,j} = \frac{\exp(S_{i,j})}{\sum_{\ell=0}^{N-1} \exp(S_{i,\ell})}.$$

The output is:

$$O_{i,r} = \sum_{j=0}^{N-1} P_{i,j} V_{j,r}.$$

The key challenge is that the softmax normalization denominator is a reduction over the full key dimension. A tiled attention kernel must preserve numerical stability while processing key blocks.

Online Softmax

A numerically stable softmax uses the row maximum:

$$m_i = \max_j S_{i,j}.$$

Then:

$$P_{i,j} = \frac{\exp(S_{i,j} - m_i)}{\sum_{\ell} \exp(S_{i,\ell} - m_i)}.$$

For blockwise processing, maintain running statistics. If a new block has local maximum m_{new} and local denominator ℓ_{new} , and the previous running maximum and denominator are m_{old} and ℓ_{old} , then:

$$m = \max(m_{\text{old}}, m_{\text{new}}),$$

$$\ell = \ell_{\text{old}} \exp(m_{\text{old}} - m) + \ell_{\text{new}} \exp(m_{\text{new}} - m).$$

The accumulated output must also be rescaled:

$$o = o_{\text{old}} \exp(m_{\text{old}} - m) + o_{\text{new}} \exp(m_{\text{new}} - m).$$

This is the mathematical foundation behind FlashAttention-style blockwise attention.

Code Walkthrough

Vector Add

A simple Helion-style vector add is:

```
@helion.kernel
def add(x, y):
    x, y = torch.broadcast_tensors(x, y)
    out = torch.empty_like(x)

    for tile in hl.tile(out.numel()):
        out[tile] = x[tile] + y[tile]

    return out
```

Explanation

- `torch.broadcast_tensors` runs in the host region and determines a common logical shape.
- `torch.empty_like` allocates the output tensor.
- `hl.tile(out.numel())` defines a tiled one-dimensional iteration space.
- The expression `x[tile] + y[tile]` is compiled into device code.
- Helion generates the Triton kernel and launcher code.
- Boundary masks are generated automatically when the final tile is partial.

The generated low-level logic is conceptually:

$$\text{offsets} = pB + [0, 1, \dots, B - 1],$$

$$\text{mask} = \text{offsets} < N.$$

Then each program performs:

$$\text{out}[\text{offsets}] = x[\text{offsets}] + y[\text{offsets}].$$

Hardware Behavior

The kernel is memory-bound. For FP32, each element requires reading x and y and writing out :

$$\text{bytes per element} = 4 + 4 + 4 = 12.$$

The arithmetic per element is one addition. Thus:

$$I = \frac{1}{12} \text{ FLOPs/byte.}$$

This is far below the arithmetic intensity needed to saturate tensor cores or CUDA cores. Performance depends mostly on memory coalescing, memory bandwidth, launch overhead, and vectorization.

Visual Concept

Draw a one-dimensional array split into equal-size tiles. Show each tile assigned to a separate GPU program. The final tile should extend past the logical end of the array, with a mask preventing out-of-bounds reads and writes.

Matrix Multiplication with Epilogue

A Helion matrix multiplication can include an epilogue function such as ReLU:

```
@helion.kernel
def matmul_epilogue(a, b, epilogue):
    M, K = a.shape
    K2, N = b.shape
    assert K == K2

    out = torch.empty((M, N), device=a.device, dtype=a.dtype)

    for tile_m, tile_n in hl.tile((M, N)):
```

```

acc = torch.zeros((tile_m, tile_n), device=a.device, dtype=torch.float32)

for tile_k in hl.tile(K):
    acc = acc + torch.matmul(a[tile_m, tile_k], b[tile_k, tile_n])

out[tile_m, tile_n] = epilogue(acc)

return out

```

Explanation

The lecture describes Helion kernels accepting a lambda or callable epilogue. For example, the user can pass ReLU:

$$\text{ReLU}(x) = \max(x, 0).$$

Then the generated kernel fuses matrix multiplication and the activation:

$$C = \text{ReLU}(AB).$$

This avoids writing the intermediate matrix AB to global memory and then launching a second kernel for ReLU.

Hardware Behavior

The outer tile loop parallelizes over output blocks. The inner reduction loop over K is sequential within each program. The accumulator stays in registers:

$$\text{acc}_{i,j} = \sum_k A_{i,k} B_{k,j}.$$

If the backend maps the operation to tensor cores, the kernel can achieve high throughput. The auto-tuner searches over tile sizes and backend options to balance:

- tensor core utilization;
- register pressure from the accumulator tile;
- memory reuse of A and B tiles;
- occupancy;
- cache behavior.

Concatenation with Masks

The lecture discusses concatenating two two-dimensional tensors along one dimension. Suppose:

$$A \in \mathbb{R}^{M \times N_1}, \quad B \in \mathbb{R}^{M \times N_2}.$$

The output is:

$$C \in \mathbb{R}^{M \times (N_1 + N_2)}.$$

The definition is:

$$C_{i,j} = \begin{cases} A_{i,j}, & j < N_1, \\ B_{i,j-N_1}, & j \geq N_1. \end{cases}$$

A Helion-style implementation is:

```
@helion.kernel
def concat_dim1(a, b):
    M, N1 = a.shape
    M2, N2 = b.shape
    assert M == M2

    out = torch.empty((M, N1 + N2), device=a.device, dtype=a.dtype)

    for tile_m, tile_n in hl.tile((M, N1 + N2)):
        from_a = tile_n < N1

        a_val = hl.load(a[tile_m, tile_n], mask=from_a, other=0.0)
        b_val = hl.load(b[tile_m, tile_n - N1], mask=~from_a, other=0.0)

        out[tile_m, tile_n] = torch.where(from_a, a_val, b_val)

    return out
```

Explanation

The output tile spans the combined second dimension. Some elements belong to A , and some belong to B . Directly indexing both tensors with the same tile would go out of bounds. Masks ensure that only valid elements are loaded.

Hardware Behavior

This kernel is mostly memory movement. Its performance depends on:

- coalesced loads from A and B ;
- coalesced stores into C ;
- avoiding unnecessary loads on masked-out regions;
- minimizing branch or predicate overhead.

RMSNorm: Straightforward Version

A simple RMSNorm kernel can be expressed close to PyTorch:

```
@helion.kernel
def rms_norm(x, weight, eps: float):
    M, N = x.shape
    out = torch.empty_like(x)

    for tile_m in hl.tile(M):
        row = x[tile_m, :]
        mean_square = torch.mean(row * row, dim=-1)
        inv_rms = torch.rsqrt(mean_square + eps)
        out[tile_m, :] = row * inv_rms[:, None] * weight[None, :]

    return out
```

Explanation

This expresses the RMSNorm formula directly:

$$y_{i,j} = x_{i,j} \cdot \text{rsqrt} \left(\frac{1}{N} \sum_{\ell=0}^{N-1} x_{i,\ell}^2 + \epsilon \right) \cdot w_j.$$

It is easy to read, but if the row length N is large, loading the entire row at once may be inefficient or impossible due to register pressure.

RMSNorm: Rolled Reduction Version

The lecture describes a more explicit version using a registered block size for inner reductions:

```
@helion.kernel
def rms_norm_rolled(x, weight, eps: float):
    M, N = x.shape
    out = torch.empty_like(x)

    rb = hl.register_block_size("reduction_block")

    for tile_m in hl.tile(M):
```

```

sumsq = torch.zeros((tile_m,), device=x.device, dtype=torch.float32)

for tile_n in hl.tile(N, block_size=rb):
    x_val = hl.load(x[tile_m, tile_n], eviction_policy="evict_last")
    sumsq = sumsq + torch.sum(x_val * x_val, dim=-1)

inv_rms = torch.rsqrt(sumsq / N + eps)

for tile_n in hl.tile(N, block_size=rb):
    x_val = hl.load(x[tile_m, tile_n], eviction_policy="evict_first")
    w_val = hl.load(weight[tile_n], eviction_policy="evict_first")
    out[tile_m, tile_n] = x_val * inv_rms[:, None] * w_val

return out

```

Explanation

The registered block size tells Helion that multiple inner loops should share the same block-size choice. This is useful when the first loop computes a reduction and the second loop writes normalized output over the same dimension.

The first inner loop computes:

$$s_i = \sum_j x_{i,j}^2.$$

The second inner loop computes:

$$y_{i,j} = x_{i,j} \cdot \text{rsqrt}\left(\frac{s_i}{N} + \epsilon\right) \cdot w_j.$$

Hardware Behavior

The first pass reads x and accumulates a scalar per row. The second pass rereads x and reads w . Cache hints express the expected reuse:

- In the first pass, x may be marked `evict_last` because it may be useful again.
- In the final use, x and w may be marked `evict_first`.

This example demonstrates Helion’s philosophy: start with a high-level expression, but allow expert users to add lower-level hints when needed.

Attention Kernel

The attention formula is:

$$O = \text{softmax} \left(\frac{QK^T}{\sqrt{d}} \right) V.$$

A simplified Helion-style attention kernel is:

```
@helion.kernel
def attention(q, k, v):
    B, H, M, D = q.shape
    _, _, N, _ = k.shape
    DV = v.shape[-1]

    out = torch.empty((B, H, M, DV), device=q.device, dtype=q.dtype)

    # Parallel over batch-head and query blocks.
    for tile_bh, tile_m in hl.tile((B * H, M)):
        q_tile = q.reshape(B * H, M, D)[tile_bh, tile_m, :]

        m_i = torch.full((tile_m,), -float("inf"), device=q.device, dtype=torch.float32)
        l_i = torch.zeros((tile_m,), device=q.device, dtype=torch.float32)
        acc = torch.zeros((tile_m, DV), device=q.device, dtype=torch.float32)

        # Sequential over key/value blocks.
        for tile_n in hl.tile(N):
            k_tile = k.reshape(B * H, N, D)[tile_bh, tile_n, :]
            v_tile = v.reshape(B * H, N, DV)[tile_bh, tile_n, :]

            scores = torch.matmul(q_tile, k_tile.transpose(-1, -2)) / math.sqrt(D)

            m_new = torch.maximum(m_i, torch.max(scores, dim=-1).values)
            p = torch.exp(scores - m_new[:, None])

            alpha = torch.exp(m_i - m_new)
            l_new = l_i * alpha + torch.sum(p, dim=-1)

            acc = acc * alpha[:, None] + torch.matmul(p, v_tile)

            m_i = m_new
            l_i = l_new

        out.reshape(B * H, M, DV)[tile_bh, tile_m, :] = acc / l_i[:, None]

    return out
```

Explanation

The important scheduling decision is:

parallel dimensions = (BH, M) , sequential reduction dimension = N .

The outer tile loop parallelizes over batch-head groups and query positions. The inner loop scans over key/value blocks because softmax requires a reduction over all keys for each query.

Hardware Behavior

Each program handles a block of queries for one or more batch-head groups. It repeatedly loads blocks of K and V , computes partial scores, updates online softmax statistics, and accumulates output. The accumulator remains in registers.

The main bottlenecks are:

- memory bandwidth for reading K and V ;
- tensor core utilization for QK^T and PV matrix multiplications;
- register pressure from score tiles, accumulators, and softmax statistics;
- occupancy limitations;
- numerical stability of online softmax.

Visual Concept

Draw the query block on the left and a sequence of key/value blocks on the right. Show the program scanning across key/value blocks while maintaining running maximum m_i , denominator ℓ_i , and output accumulator o_i .

Compiler Internals

Python-Embedded DSL

Helion is embedded in Python. The compiler starts from a Python abstract syntax tree, or AST. The rough pipeline is:

Python AST $\rightarrow$ typed and annotated AST $\rightarrow$ FX graphs $\rightarrow$ Inductor IR $\rightarrow$ compiler passes $\rightarrow$ Triton code generator

Basic Blocks

The lecture explains that Helion splits code into **basic blocks**. A basic block is a sequence of instructions with no internal control-flow branch. For example, an `if` statement creates different basic blocks for the then-branch and else-branch.

This matters because compilers often handle straight-line code separately from control-flow structure. Each basic block can be converted into an FX graph, and then cross-block control flow can be handled at a higher level.

Reuse of TorchInductor

Helion reuses TorchInductor, the same backend used by `torch.compile`. However, it does not reuse all parts of `torch.compile`.

It reuses:

- PyTorch operator lowering;
- Inductor IR machinery;
- Triton code generation infrastructure.

It does not need to reuse graph-level fusion decisions, because a Helion kernel is already defined by the user as one kernel.

`torch.compile` : decide fusion boundaries automatically.

Helion : the user writes the fusion boundary explicitly.

When Configuration Is Applied

The lecture notes that auto-tuning configuration is applied late in the pipeline, during code generation. This is important because the earlier compiler pipeline can be run once, while code generation can be repeated many times for different configurations during auto-tuning.

Conceptually:

$$\underbrace{\text{parse} \rightarrow \text{type} \rightarrow \text{FX} \rightarrow \text{IR}}_{\text{run once}} \rightarrow \underbrace{\text{codegen}(c) \rightarrow \text{benchmark}}_{\text{run for many configs}}.$$

Generated Output

Helion generates Python Triton code plus wrapper code. The wrapper is produced by rewriting the AST around the `hl.tile` loop, replacing the high-level loop with a kernel call.

This generated code resembles TorchInductor output because of the shared backend stack.

Debugging and Development Workflow

Printing Generated Code

Helion provides an environment variable similar in spirit to `TORCH_LOGS=output_code`. The lecture mentions:

`HELION_PRINT_OUTPUT_CODE=1`

This prints the generated Triton code and launcher code. It is useful for:

- understanding what the auto-tuner selected;
- debugging numerical issues;
- creating minimal reproducers;
- learning how Helion maps high-level tiles to Triton offsets and masks.

Disabling Auto-Tuning During Development

Full auto-tuning may be slow. During development, the user may only care whether the kernel is correct. The lecture mentions:

`HELION_AUTOTUNE_EFFORT=none`

This selects a default configuration without running the full auto-tuner.

Interpreted Mode

The lecture also mentions an interpreted mode:

`HELION_INTERPRET=1`

In interpreted mode, Helion runs the kernel logic through PyTorch eager execution rather than the Triton backend. This helps debug correctness by comparing high-level semantics against generated code.

Logging

A logging mode is also described:

`HELION_LOGS=all`

or a debug-style variant such as:

`HELION_LOGS=+all`

These logs expose compiler diagnostics and are useful when investigating type propagation, lowering behavior, or code generation.

Relationship to Other Systems

Relationship to Triton

Helion currently lowers to Triton. A useful comparison is:

Triton = explicit program IDs, offsets, masks, and schedules.

Helion = PyTorch-like tiled semantics plus auto-tuned lowering.

Triton programmers explicitly decide many low-level details. Helion programmers decide higher-level tiling structure and let the auto-tuner search over the details.

Relationship to `torch.compile`

`torch.compile` takes a PyTorch program and decides how to optimize it. Helion gives the user explicit control over the kernel boundary.

$\text{torch.compile} : f_{\text{PyTorch}} \mapsto \text{optimized graph with compiler-chosen kernels.}$

$\text{Helion} : f_{\text{kernel}} \mapsto \text{one user-defined generated kernel.}$

The lecture also mentions that Helion kernels can compose with `torch.compile`. A model can contain PyTorch operations, Triton kernels, and Helion kernels. Today, Helion kernels are treated similarly to custom Triton kernels. Future work may include tighter prologue or epilogue fusion with `torch.compile`.

Relationship to Halide

The lecture explicitly connects Helion to ideas from Halide. Halide separated:

algorithm from schedule.

Helion has a similar flavor:

Helion source code $\approx$ semantic description,

auto-tune configuration $\approx$ schedule.

However, Helion integrates this idea into a PyTorch-like ML kernel authoring workflow and uses auto-tuning to search schedules.

Relationship to Low-Level Hardware-Specific DSLs

Some modern systems move toward lower-level control because hardware such as Blackwell introduces complex features that may require explicit intrinsics. Helion takes a complementary approach: keep the common language high level and rely on auto-tuning for most kernels, while preserving escape hatches for inline Triton or inline PTX when needed.

The lecture’s philosophy is not:

high-level DSLs should replace low-level programming.

Instead, it is:

use high-level DSLs for most kernels, and low-level code when absolutely necessary.

Performance Results Discussed

RMSNorm Backward

The lecture discusses RMSNorm backward comparisons against systems such as CuTe DSL and Quack. The qualitative result is that Helion performs robustly across shape ranges, while other systems may win in specific regimes.

The important engineering point is not just peak performance on one shape, but **performance portability**:

good performance across many shapes > excellent performance on one hand-tuned shape.

B200 Results

The lecture mentions B200 speedups where Helion performs well against eager PyTorch, `torch.compile`, and handwritten Triton kernels from a repository of Triton kernels. The reported geometric mean speedup over eager PyTorch is around 3.2×, and Helion is described as nearly twice as fast as handwritten Triton in that collection.

The explanation given is that the auto-tuner can search Triton code generation choices more thoroughly than most humans will manually do for every kernel and shape.

AMD Results

The lecture also mentions AMD results with a similar qualitative story: Helion’s auto-tuning gives robust performance across shapes and hardware. This supports the portability claim.

Performance Intuition

Why Helion Can Beat Handwritten Triton

At first, it may seem surprising that a higher-level DSL can outperform handwritten Triton. The reason is that handwritten Triton often represents one

programmer’s chosen point in the search space. Helion can generate many Triton variants from one source kernel.

If the search space has 1200 candidates, a human-written kernel is just one candidate:

$$K_{\text{human}} = K(c_{\text{human}}).$$

Helion searches:

$$\{K(c_1), K(c_2), \dots, K(c_{1200})\}.$$

The best candidate is:

$$K(c^*) = \arg \min_i T(K(c_i)).$$

Even if the human-written kernel is good, there may be nearby variants with better block sizes, indexing modes, launch layouts, or backend flags.

Why Auto-Tuning Helps Hardware Portability

A kernel tuned for one GPU generation may not be optimal on another. Differences include:

- number of SMs;
- register-file size;
- shared-memory capacity;
- L2 cache size;
- tensor core throughput;
- memory bandwidth;
- support for tensor descriptors or new memory instructions;
- scheduler behavior.

For hardware h , the optimal configuration is:

$$c^*(h) = \arg \min_c T(c, h).$$

A handwritten low-level kernel may encode assumptions about h_{old} . Helion can retune for h_{new} :

$$c^*(h_{\text{old}}) \neq c^*(h_{\text{new}}).$$

When Helion Is Most Useful

Helion is especially useful when:

- the kernel is custom enough that PyTorch eager is slow;
- `torch.compile` does not produce the desired fusion;
- handwritten Triton would require tedious indexing and tuning;
- the kernel must run across multiple shapes or hardware targets;
- the programmer wants to express PyTorch-like tensor operations inside a custom kernel.

When Lower-Level Programming May Still Be Needed

Lower-level programming may still be needed when:

- the kernel depends on brand-new architecture-specific instructions;
- peak performance requires explicit PTX or assembly;
- the schedule is too specialized for the current auto-tuner;
- the computation requires synchronization patterns not yet expressible in Helion;
- the kernel is part of a large mega-kernel with complex internal scheduling.

Mega-Kernels and Future Directions

Current State

The lecture discusses whether Helion could express mega-kernels like systems that fuse large portions of a model into one kernel. Currently, Helion supports multiple top-level `hl.tile` loops in a for-each style, with some limitations.

A Helion kernel is still one generated kernel, so in principle multiple independent tiled computations can live inside it. However, complex sequencing between tiled stages would require stronger language constructs.

Potential Barrier Construct

A possible future construct is a barrier separating groups of tile loops:

tile loops before barrier $\rightarrow$ barrier $\rightarrow$ tile loops after barrier.

This would allow programmers to express that all work in one stage must complete before the next stage begins. Such a feature could make Helion more suitable for mega-kernel programming.

When Mega-Kernels Are Profitable

Mega-kernels are often most profitable for small shapes where launch overhead dominates. If each kernel does little work, the launch overhead T_{launch} becomes significant.

For L separate kernels:

$$T_{\text{separate}} = \sum_{\ell=1}^L (T_{\text{launch}} + T_{\ell}).$$

For one fused mega-kernel:

$$T_{\text{mega}} = T_{\text{launch}} + T_{\text{fused}}.$$

The potential launch-overhead savings are approximately:

$$(L - 1)T_{\text{launch}}.$$

For large kernels, computation dominates and the benefit may be smaller.

CUDA Graphs and Composability

The lecture discusses CUDA graph compatibility. Helion kernels should be CUDA graphable as long as the host-side code does not perform operations that are incompatible with CUDA graphs. Since the device region lowers to a normal Triton/CUDA kernel, graph capture is conceptually similar to capturing other GPU kernels.

A PyTorch program may contain:

PyTorch ops + Triton kernels + Helion kernels + custom CUDA kernels.

These compose through ordinary tensor inputs and outputs. Each kernel remains a separate GPU launch unless additional fusion mechanisms are introduced.

Low-Level Escape Hatches

Inline Triton and Inline PTX

Helion supports escape hatches for low-level code. The lecture mentions that users can insert inline PTX or inline Triton. This is important for new hardware features or instructions not yet exposed through higher-level APIs.

A representative pattern is:

```
def custom_fma(a, b, c):
    return hl.inline_asm_elementwise(
        asm="fma.rn.f32 $0, $1, $2, $3;",
        constraints="=f,f,f,f",
        args=[a, b, c],
        dtype=torch.float32,
        is_pure=True,
        pack=1
    )
```

Explanation

The high-level Helion kernel can remain mostly PyTorch-like, while a small inner operation uses a specific hardware instruction. This preserves productivity for most code while allowing expert control where needed.

Common Misconceptions

Misconception: Helion Is Just Triton Syntax Sugar

Helion does remove Triton boilerplate, but its key feature is the integrated auto-tuner. A Helion kernel defines a large search space of possible Triton kernels. This is deeper than simply rewriting Triton syntax.

Misconception: Helion Replaces `torch.compile`

Helion and `torch.compile` solve different problems. `torch.compile` optimizes a PyTorch graph and chooses fusion boundaries automatically. Helion lets the user explicitly author one custom kernel.

Misconception: Higher-Level Means Slower

A high-level language can be fast if it exposes the right semantic structure and searches over low-level schedules. Helion may beat handwritten Triton when the auto-tuner finds better configurations than the human-selected one.

Misconception: The User No Longer Needs Hardware Intuition

Helion removes many tedious details, but the user still needs to understand which dimensions should be parallel and which must be sequential. This is especially important for reductions, attention, scans, and kernels requiring atomics.

Misconception: All Python Works Inside the Device Region

The host region is ordinary Python. The device region is compiled to Triton and has restrictions. Some operations, such as `print`, may map to Triton equivalents. Other Python debugging tools, such as breakpoints inside device code, are not generally available in the same way.

Misconception: Auto-Tuning Is Free

Auto-tuning can be expensive. The intended workflow is to tune representative shapes, save the resulting configurations, and reuse them.

Practical Takeaways

1. Helion is best understood as PyTorch-like tiled kernel programming.
2. The outermost `hl.tile` loop defines parallel work.
3. Inner loops are sequential within a generated program.
4. One Helion kernel lowers to one Triton kernel.
5. Helion reuses TorchInductor for PyTorch operator lowering.
6. The auto-tuner searches over indexing modes, block sizes, loop flattening, program-ID strategies, persistent grids, backend flags, masking, and reduction strategies.
7. Reduction rolling is essential when a naive reduction would load too much data into registers.
8. Performance portability is a major goal: retune rather than rewrite for new hardware.
9. Debugging tools include generated-code printing, interpreted execution, logging, and disabling auto-tuning during development.
10. Helion does not remove the need for GPU intuition; it changes the level at which that intuition is applied.

Project or Experiment Ideas

Experiment 1: Add Kernel Bandwidth

Implement vector add in PyTorch eager, `torch.compile`, Triton, and Helion. Measure effective bandwidth:

$$B_{\text{eff}} = \frac{12N}{T}$$

for FP32 vector add, where T is runtime in seconds.

Experiment 2: Naive Versus Rolled RMSNorm

Write two RMSNorm kernels:

- one that expresses the reduction directly with `torch.mean`;
- one that manually rolls the reduction over smaller tiles.

Measure runtime as N increases and observe where register pressure or poor occupancy appears.

Experiment 3: Attention Parallelization Choices

Implement a simple attention kernel and vary which dimensions are placed in the outer `hl.tile` loop. Compare:

(BH, M) -parallel versus (BH, M, N) -parallel with atomics.

Analyze why the softmax reduction dimension is naturally sequential unless a more complex parallel reduction strategy is used.

Experiment 4: Auto-Tuning Shape Sensitivity

Tune the same Helion kernel for small, medium, and large shapes. Record the best configurations:

$$c_{\text{small}}^*, \quad c_{\text{medium}}^*, \quad c_{\text{large}}^*.$$

Compare block sizes, indexing modes, and backend flags.

Experiment 5: Inspect Generated Triton

Use generated-code printing and study how Helion lowers:

- `hl.tile`;
- masks;
- PyTorch operations;
- reductions;
- launch grids.

Then manually modify the generated Triton code to test whether a human can improve or break the tuned result.

Experiment 6: Cache Eviction Policies

Use RMSNorm or LayerNorm and experiment with eviction policies. Measure whether `evict_last` helps when an input is reused and whether `evict_first` helps for one-time-use tensors.

Final Mental Model

Helion is a kernel-authoring DSL that asks the programmer to think at the level of **tiled tensor semantics** rather than explicit pointer arithmetic. The programmer decides what computation belongs in one kernel and which dimensions are parallel versus sequential. Helion then lowers the kernel through PyTorch compiler infrastructure into Triton and uses auto-tuning to search over the low-level schedule.

The deepest idea from the lecture is:

Write the computation once at a tiled PyTorch level; search the implementation space automatically.

This turns many kernel-engineering problems from manual schedule design into configuration search. It does not eliminate the need for GPU systems understanding, but it moves the engineer's effort toward higher-leverage decisions: parallelization structure, reduction structure, fusion boundary, memory reuse, and when to use low-level escape hatches.